Programming Language Concepts Project Report

Student Name	Student ID	Student Email
Selina Tan Hui Shan	1007741	1007741@mymail.sutd.edu.sg
Loke Jing Yang	1008156	1008156@mymail.sutd.edu.sg
Hafsah Hussain Ahamed	1008196	1008196@mymail.sutd.edu.sg
Escabas Joyce Baldos	1008181	1008181@mymail.sutd.edu.sg

1. Project Overview 4
1.1. Project Context 4
1.2. Project Management 4
1.3. Development Process 5
1.3.1. System Architecture Diagram 5
1.3.2. Finite State Machine (FSM) 5
1.3.3. Input 6
1.3.4. Lexer (Tokenizer) 7
1.3.5. Parser 7
1.3.5.1. Grammar Design and Implementation 7
1.3.5.2. Abstract Syntax Tree (AST) 9
1.3.6. Evaluation 10
1.3.6.1. Simple Calculator Mode 10
1.3.6.2. Differentiation Mode 11
1.3.7. Graph Mode 12
1.3.8. Test Cases 13
1.3.8.1 Simple Calculator Mode 13
1.3.8.2 Differentiate Mode 14
1.3.9. Design Choices 14
1.3.9.1. Overall UI/UX 14
1.3.9.2. Choice Input 14
1.3.10. Additional Notes 15
Max Integer Size Printing Outputs 15
1.4. Compatibility Measures 16
2. Constraint Satisfaction 17
2.1. File Reading/Writing 17
2.2. Parser 17
2.3. State Machine 17
2.4. External Libraries 17
2.5. C Programming Principles & Practices 18
2.5.1. Memory Management 18
2.5.2. Modularity & Reusability 18
2.5.3. Code Practices 18
2.5.3.1. Don’t Repeat Yourself (DRY) 18
2.5.3.2. Single Responsibility Principle (SRP) 18
2.5.4. C Project Practices 18
2.5.4.1. Makefile 18
2.5.4.2. README 18
3. Challenges 19
3.1. Finite State Machine (FSM) 19
3.2. Context Free Grammar (CFG) 19
3.3. Abstract Syntax Tree (AST) 19
3.4. Third-party Library for Graph in C 19

4. Conclusion.....20
5. Appendix / References..... 21
5.1. GitHub Link.....21
5.2. pbPlots GitHub Link.....21
5.3. Table 1: Simple Calculation Test Cases..... 21
5.4. Table 2: Differentiate Test Cases.....23

1. Project Overview

1.1. Project Context

The project focuses on the development of a program using the C programming language. The problem statement we have chosen to address is: “How might we design and implement a graphing calculator in the C programming language that can parse, evaluate, and visually plot mathematical expressions?” This task requires several functionalities, such as the following:

- Handling of user input
- Interpreting mathematical syntax
- Evaluating the mathematical input
- Generating a graphical output

The objective is then to develop a graphing calculator program that is capable of accepting user-defined mathematical expressions, processing them and depending on the user’s needs, either evaluating, differentiating, or plotting the respective graphs. As such, to achieve this, we have decomposed the project into the following components:

- Input Manager
- Lexer (Tokenizer)
- Parser
- Evaluator
- Graph Manager

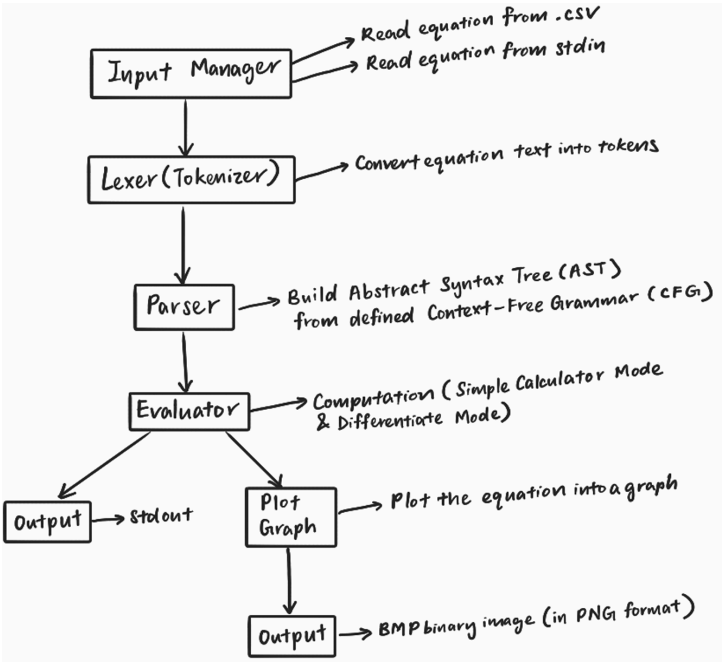
Each component is responsible for a specific stage of the computation pipeline, ranging from input handling and lexical analysis to expression evaluation and graph rendering. These components and their implementations are discussed further in detail in the subsequent sections of this report.

1.2. Project Management

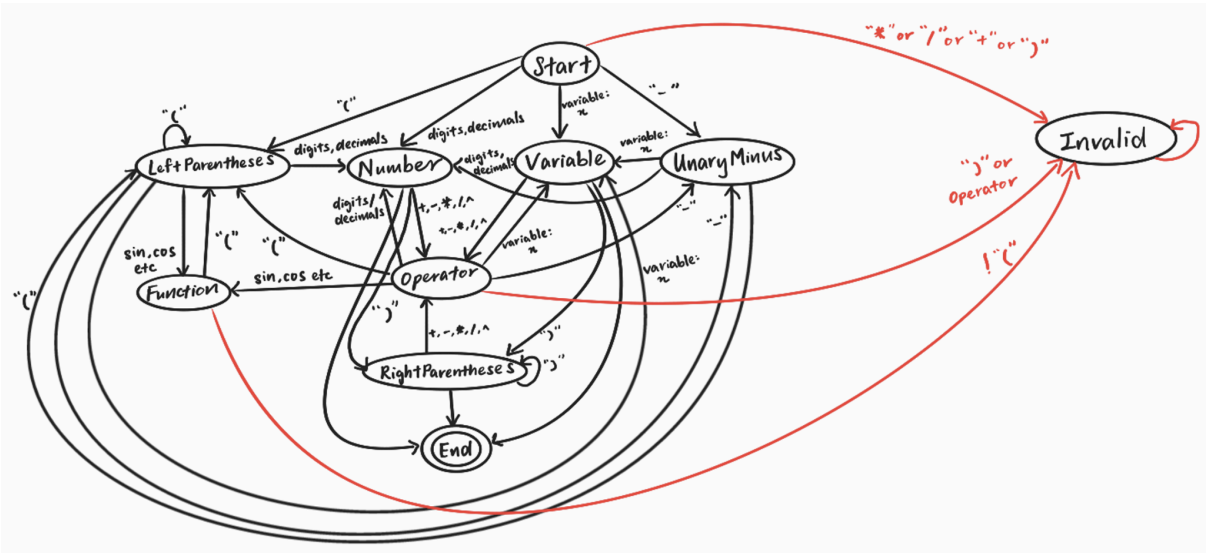
Name	Task Distribution
Selina Tan Hui Shan	Finite State Machine, Context-Free Grammar, Abstract Syntax Tree, Evaluation
Loke Jing Yang	System Architecture Diagram, Graph Manager
Hafsah Hussain Ahamed	Input Manager, Lexer (Tokenizer)
Escabas Joyce Baldos	Makefile, Test Cases, UI/UX, Design Choices

1.3. Development Process

1.3.1. System Architecture Diagram



1.3.2. Finite State Machine (FSM)



States:

- $Function$: $sin,\ cos,\ tan,\ exp,\ log$
- $Number$: $integers,\ floats$
- $Variable$: x
- $Operator$: $+,\ -,\ *,\ /,\ ^\wedge$
- $LeftParentheses$: $($
- $RightParentheses$: $)$
- $UnaryMinus$: $-$

State	Purpose	Valid Next Transitions
Start	Initial State	LeftParentheses, Number, Variable, UnaryMinus
Number	Parsing Integer/Float	Operator, RightParentheses, End
Variable	Parsing x	Operator, RightParentheses, End
Operator	Parsing $+,\ -,\ *,\ /,$ ^	Number, Variable, LeftParentheses, Function, UnaryMinus
Function	Parsing $sin,\ cos,\ tan,\ exp,\ log$	LeftParentheses
LeftParentheses	(- open parentheses	Number, Variable, Function, LeftParentheses, UnaryMinus
RightParentheses	) - close parentheses	Operator, RightParentheses, End
UnaryMinus	–	Number, Variable, LeftParentheses

1.3.3. Input

The input stage begins in main.c, where the program runs a menu loop offering 4 different modes:

1. Simple Calculator
2. Differentiate
3. Show Graphs
4. Exit the program.

The menu is printed via the printMenu() function, and the user's selection is read with fgets into a small buffer. The input is validated by checking that exactly one digit was entered. Anything outside the range 0-4 is rejected, and the prompt is repeated. Once the valid choice is made, printSelectedChoice() prints a header banner for the selected mode before the equation input begins.

Furthermore, there are additional cases, determined by whether the programme is run using a file. e.g. ./calculator eq.csv enables 'file mode'.

1. In stdin mode, after selecting a mode from the menu, the user is prompted to enter an equation directly. fgets reads the input into a 256-character buffer, and strcspn strips the trailing newline before the string is forwarded for processing.

2. In file mode, a file path is passed as a command line argument. The program opens the file and reads it line by line with fgets. Each line can hold multiple equations separated by commas like e.g : "x+1, x^2". Strtok is used to split on commas, with each segment trimmed individually before being processed. Mode 0 (Run Tests) is disabled in file mode since test cases are defined internally, not read from a file.

In both modes, input is passed to process_equation() in helper.c, which trims whitespace and applies mode-specific preprocessing before tokenizing. For mode 2, if the equation contains “=” (e.g., y = x^2), the LHS is stripped so that only the RHS is tokenized. For mode 1, if the token stream contains any single letter identifier, the equation is rejected since variables are not allowed in simple calculator mode.

1.3.4. Lexer (Tokenizer)

The tokenizer in the lexer.c takes the cleaned-up equation string and breaks it down into a list of tokens, where each token has a type and the raw text it came from. It reads through the string one character at a time and decides what each character or group of characters represents. A marker token is added at the end so that the parser knows when the list is finished.

The classification works as follows:

Input	Token Type	Notes
Digit or ‘.’	TOKEN_NUMBER / TOKEN_INVALID	Validated by is_valid_number(). At most one decimal point is valid 3.14
Letter	TOKEN_IDENTIFIER / TOKEN_NUMBER / TOKEN_INVALID	The full alphanumeric run is collected, so x1 counts as one token. The parser later decides if it is a function name like 'sin' or a variable like x.
+ - / * ^	TOKEN_OPERATOR	Each symbol is treated as its own token
(	TOKEN_LPAREN	Opens a grouped expression or a function call
)	TOKEN_RPAREN	Closes a grouped expression or a function call
,	TOKEN_COMMA	Separates arguments in a function call
anything else	TOKEN_INVALID	Any unrecognised character is flagged as invalid

1.3.5. Parser

1.3.5.1. Grammar Design and Implementation

To define the language accepted by the calculator, we first designed a theoretical Context-Free Grammar (CFG) and subsequently refined it to align with the requirements of the recursive descent parser implementation.

Initial Draft for Context-Free Grammar (CFG)

The initial grammar defines the syntactic structure, operator precedence, and associativity of the supported mathematical language:

1. $Start \rightarrow Expression$
2. $Expression \rightarrow Expression + Term \mid Expression - Term \mid Term$
3. $Term \rightarrow Term * Power \mid Term / Power \mid Power$
4. $Power \rightarrow UnaryMinus \text{ ^ } Power \mid UnaryMinus$
5. $UnaryMinus \rightarrow - UnaryMinus \mid Factor$
6. $Factor \rightarrow Number \mid Variable \mid Function(Expression) \mid (Expression)$
7. $Function \rightarrow sin \mid cos \mid tan \mid exp \mid log$
8. $Variable \rightarrow x$
9. $Number \rightarrow Integer \mid Integer \text{ "." } Digits \mid \text{"." } Digits$
10. $Integer \rightarrow Digit\ Integer \mid Digit$
11. $Digits \rightarrow Digit\ Digits \mid Digit$
12. $Digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

The initial CFG provides a clear hierarchical structure for mathematical expressions, including operator precedence and function support. However, several issues were identified during implementation.

Firstly, the grammar contains direct left recursion in the following rules:

1. $Expression \rightarrow Expression + Term \mid Expression - Term \mid Term$
2. $Term \rightarrow Term * Power \mid Term / Power \mid Power$
3. $Power \rightarrow UnaryMinus \text{ ^ } Power \mid UnaryMinus$

Left recursion is incompatible with recursive descent parsers, as implemented in ast.c, because it leads to infinite recursion and stack overflow.

Secondly, although the grammar correctly expresses mathematical structure, it does not reflect the iterative parsing strategy used in the implementation. In particular, left-associative operators $(+, -, *, /)$ are handled using iterative loops in the parser rather than recursive productions.

Final Context-Free Grammar (CFG)

To resolve these issues, the grammar was transformed to eliminate left recursion and align with recursive descent parsing:

1. $Start \rightarrow Expression$
2. $Expression \rightarrow Term\ Expression'$
3. $Expression' \rightarrow + Term\ Expression' \mid - Term\ Expression' \mid \varepsilon$
4. $Term \rightarrow Power\ Term'$
5. $Term' \rightarrow * Power\ Term' \mid / Power\ Term' \mid \varepsilon$
6. $Power \rightarrow UnaryMinus\ Power'$
7. $Power' \rightarrow \text{ ^ } UnaryMinus\ Power' \mid \varepsilon$
8. $UnaryMinus \rightarrow - UnaryMinus \mid Factor$
9. $Factor \rightarrow Number \mid Variable \mid Function(Expression) \mid (Expression)$
10. $Function \rightarrow sin \mid cos \mid tan \mid exp \mid log$
11. $Variable \rightarrow x$
12. $Number \rightarrow Integer \mid Integer \text{ "." } Digits \mid \text{"." } Digits$
13. $Integer \rightarrow Digit\ Integer \mid Digit$

14. $Digits \rightarrow Digit\ Digits \mid Digit$
15. $Digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

The revised grammar eliminates left recursion through the introduction of helper non-terminals ($Expression'$, $Term'$ and $Power'$). The structure is compatible with recursive descent parsing. It also preserves the correct operator precedence: "^" has the highest precedence and is right-associative, "*" and "/" have intermediate precedence and are left-associative, while "+" and "-" have the lowest precedence and are left-associative.

Implementation Constraints (Semantic Rules)
Beyond the syntactic rules defined by the CFG, additional semantic rules are enforced during parsing and evaluation:

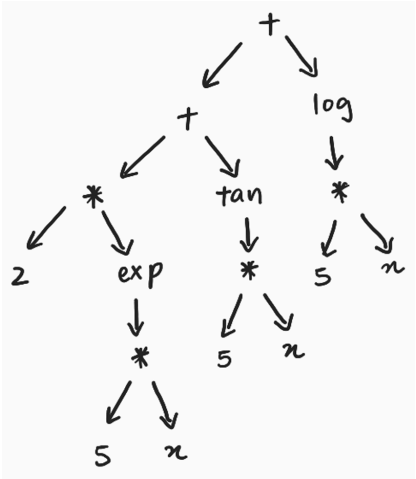
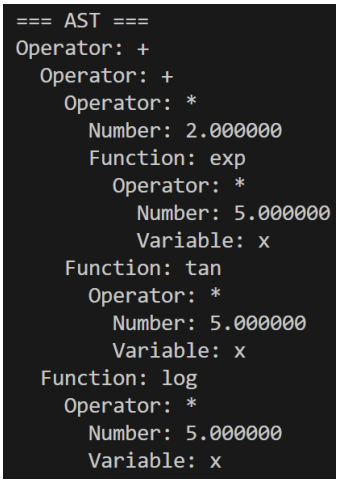
1. Mode-dependent variable usage: In $Simple\ Calculator\ mode$, variables such as x are not allowed. This is enforced during the $parseFactor()$ phase.
2. Function argument restrictions: In $Simple\ Calculator\ mode$, function arguments must be numeric constants and cannot contain variables. This is validated using the $containsVariable()$ helper function.
3. Identifier Validation: While the lexer accepts general alphanumeric identifiers, the parser restricts valid identifiers to predefined function names $(sin,\ cos,\ tan,\ exp,\ log)$ and the variable x. Any other identifier results in a parsing error.

1.3.5.2. Abstract Syntax Tree (AST)

Rather than directly implementing the CFG as a final structure, it serves as a blueprint for constructing an Abstract Syntax Tree (AST).

The AST removes non-semantic syntactic elements, such as intermediate non-terminals and parentheses, retaining only semantically meaningful operations. This representation enables efficient evaluation and transformation of expressions.

Example
An example is the expression $2 * exp(5 * x) + tan(5 * x) + log(5 * x)$, the visual representation of the AST and parser output is shown below:

	
Visual Representation of AST	Parser Output

```
=== AST ===
Operator: +
  Operator: +
    Operator: *
      Number: 2.000000
      Function: exp
        Operator: *
          Number: 5.000000
          Variable: x
    Function: tan
      Operator: *
        Number: 5.000000
        Variable: x
  Function: log
    Operator: *
      Number: 5.000000
      Variable: x
```

1.3.6. Evaluation

1.3.6.1. Simple Calculator Mode

The evaluation component computes the numerical result of an expression by traversing the Abstract Syntax Tree (AST). This is implemented in the $evaluate_expression()$ function within $evaluator.c$.

The evaluation follows a recursive approach, where each node in the AST is processed based on its type:

- Number nodes (n) return their stored numeric value.
- Binary operator nodes (+, -, *, /, ^) recursively evaluate their left and right subtrees and apply the corresponding arithmetic operation.
- Unary minus nodes (u) evaluate their child node and negate the result.
- Function nodes (f) evaluate their argument and apply the corresponding mathematical function $(sin,\ cos,\ tan,\ exp,\ log)$.

In Simple Calculator mode, variables are not allowed. This constraint is enforced during parsing in $parseFactor()$, ensuring that all expressions evaluated in this mode consist only of numeric constants.

Trigonometric functions are evaluated in degrees. These are converted to radians using the helper function $degree_to_radian()$, as the standard math library operates in radians. This design choice improves usability, as users typically input angles in degrees and would otherwise need to manually perform the conversion.

The evaluation process ensures that the structure of the AST determines the order of computation, preserving operator precedence and associativity defined during parsing.

1.3.6.2. Differentiation Mode

In differentiation mode, the system supports both symbolic differentiation and automatic differentiation using dual numbers.

Symbolic differentiation is implemented in the $differentiate()$ function, which constructs a new AST representing the derivative of the input expression. The function applies standard differentiation rules recursively:

1. Constant rule: $\frac{d}{dx}(c) = 0$
2. Variable rule: $\frac{d}{dx}(x) = 1$
3. Constant multiple rule: $\frac{d}{dx}(c * f(x)) = c * f'(x)$
4. Addition: $\frac{d}{dx}(f(x) + g(x)) = f'(x) + g'(x)$
5. Subtraction: $\frac{d}{dx}(f(x) - g(x)) = f'(x) - g'(x)$
6. Product rule: $\frac{d}{dx}(f(x) * g(x)) = f'(x) * g(x) + f(x) * g'(x)$
7. Quotient rule: $\frac{d}{dx}(\frac{f(x)}{g(x)}) = \frac{g(x)f'(x) - f(x)g'(x)}{(g(x))^2}$
8. Power rule:
    a. For constant exponent: $\frac{d}{dx}((f(x))^c = c * (f(x))^{(c-1)} * f'(x)$
    b. For the general case:
    $\frac{d}{dx}((f(x))^{g(x)} = (f(x))^{g(x)} * (g'(x) * ln(f(x)) + g(x) * \frac{f'(x)}{f(x)})$
9. Function rules:
    a. Sine: $\frac{d}{dx}(sin(f(x))) = cos(f(x)) * f'(x)$
    b. Cosine: $\frac{d}{dx}(cos(f(x))) = -sin(f(x)) * f'(x)$
    c. Tangent: $\frac{d}{dx}(tan(f(x))) = \frac{1}{(cos(f(x)))^2} * f'(x)$
    d. Exponential: $\frac{d}{dx}(exp(f(x))) = exp(f(x)) * f'(x)$
    e. Logarithm (Base 10): $\frac{d}{dx}(log10(f(x))) = \frac{1}{f(x)*ln(10)} * f'(x)$

After differentiation, the resulting AST is simplified using the $simplify()$ function. This performs optimisation such as constant folding and removal of identity elements (eg. $x + 0 = x, x * 1 = x$).

Automatic differentiation is implemented in $dualUtils.c$ using dual numbers. A dual number is represented as $a + b\epsilon$, where a is the real value, b is the derivative, $\epsilon \neq 0$ and $\epsilon^2 = 0$.

Each arithmetic and mathematical operation is extended to operate on dual numbers:

1. Addition: $(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$
2. Subtraction: $(a + b\epsilon) - (c + d\epsilon) = (a - c) + (b - d)\epsilon$
3. Multiplication: $(a + b\epsilon)(c + d\epsilon) = a * c + (a * d + b * c)\epsilon$
4. Division: $\frac{(a+b\epsilon)}{(c+d\epsilon)} = \frac{a}{c} + \frac{b*c-a*d}{c^2}\epsilon$
5. Power: $(a + b\epsilon)^{(c+d\epsilon)} = a^c * (1 + (d * ln(a) + \frac{c*b}{a})\epsilon)$

6. Functions:
    - Sine: $sin(a + b\epsilon) = sin(a) + b * cos(a)\epsilon$
    - Cosine: $cos(a + b\epsilon) = cos(a) - b * sin(a)\epsilon$
    - Tangent: $tan(a + b\epsilon) = tan(a) + b * \frac{1}{cos^2(a)}\epsilon$
    - Exponential: $exp(a + b\epsilon) = exp(a) + b * exp(a)\epsilon$
    - Logarithm (Base 10): $log(a + b\epsilon) = log(a) + \frac{b}{a}\epsilon$

The evaluation function $evaluate_expression_dual()$ computes expressions by initialising the variable x as $(x, 1)$. The real component represents the function value $f(x)$, while the dual component represents the derivative $f'(x)$. This approach enables computation of derivatives in a single forward pass without explicitly constructing a symbolic derivative.

1.3.7. Graph Mode

The graphing component of the program is responsible for processing a user-provided equation and generating a PNG image that visualises the corresponding graph.

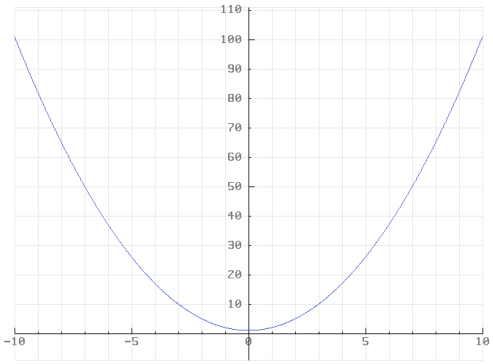
The core function of this component, $plot_graph()$, begins by accepting the Abstract Syntax Tree (AST) representation of the input equation. This structured form allows efficient evaluation of the mathematical expression across a range of values.

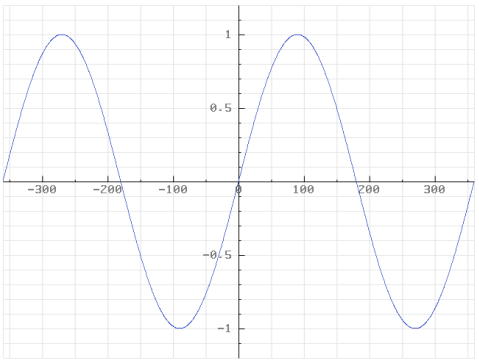
The function first determines an appropriate range for the x-axis. It checks whether the equation contains any trigonometric functions using $contains_trig_functions()$. If trigonometric terms are detected, the x-range is adjusted to display two full periods of the function to better capture its periodic behaviour. Otherwise, a default range is applied.

Next, the y-axis range is computed dynamically. Initially set to a broad range, the function iterates over the selected x-values and evaluates the corresponding y-values using $evaluate_expression()$. During this process, $is_valid_point()$ is used to identify discontinuities or asymptotic behaviour by detecting sudden large changes in y-values. Only valid points are considered, and the minimum and maximum of these values are used to determine an appropriate y-axis range.

Finally, the graph is rendered using a third-party library, pbPlots (link to original source code under 5.2. pbPlots GitHub Link:). The function configures the graph's visual properties, including axes, labels, and plot styling, before generating and saving the resulting image as a PNG file on the user's local device.

The following are some examples of graphs generated with this program:





1.3.8. Test Cases

Test cases and scenarios were implemented to pinpoint potential calculation errors, which can be used to debug specific points of our code. The main tests performed pertained only to the simple calculator mode, which was the biggest point of concern. These tests can be toggled via inputting ‘0’ as a choice, if not in file mode.

Tests and test cases were implemented similarly to CFG functions, primarily for initialisation and memory handling. Structs $test$ included the members $EvalTestType,\ eqn,\ ans$ which stores the test type, equation string and its expected answer. This allowed the potential for testing within other modes. Guidelines were implemented during the creation of unit tests for both the simple calculator and the differentiation mode to ensure that important edge cases were hit but not necessarily repeated.

1.3.8.1 Simple Calculator Mode

Each potential sequence of operations are separated into “Singular OPs” and “Compound OPs”. The related test cases may be found in Table 1 of the Appendix.

- **Singular OPs**
  (e.g. Addition, Subtraction…etc)
  - Evaluate MAX INTEGERS/LONG
  - Tackle basic units
- **Compound OPs**
  (e.g. Both Addition & Subtraction…etc)
  - For Inverse OPs (Additive/Multiplicative)
    - MAX INTEGERS/LONGS
    - Try not to repeat the cases from singular OP test cases (handle only the most basic combinations because complex combinations would already be handled by the singular OPs test cases)
  - For Combined OPs
    - Test Cases should primarily be testing order

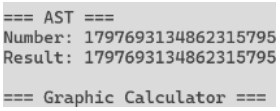
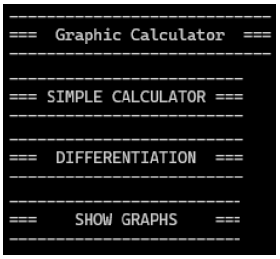
1.3.8.2 Differentiate Mode

Since the test cases for simple calculator mode already handle checking of the basic evaluation operations, the tests within this mode need not be as extensive. However, a few test cases were curated **'Singular OPs'** to double check how the x variable is handled as it is not a terminal within simple calculation. The related test cases may be found in Table 2 of the Appendix.

1.3.9. Design Choices

1.3.9.1. Overall UI/UX

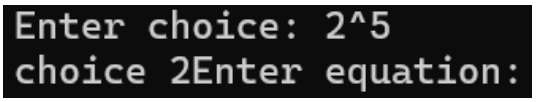
Formatting of printed statements were changed throughout the course of development for better readability as shown in the table below.

Initial Design		Finalized Design	
Only headers per module printed using the format: "=== {name} ===" also used by the menu		Headers are now separated based on hierarchy using blocks and length; Initial format is still kept for sub-headers pertaining to tokenizer & AST	
No separator between equations when evaluating a file of equations		Equations separated by lines	

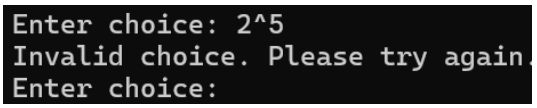
1.3.9.2. Choice Input

During prompt to enter choice In previous iterations, user inputs such as "1a1a1a1" or "2^5" would default to the first numerical input before a non-numerical input is detected due to the use of $atoi()$. For better usability and readability, choice selection is strictly held to the constraint $choice \in \{0, 1, 2, 3, 4\}$.

Before:



After:



1.3.10. Additional Notes

Max Integer Size Printing Outputs

During normal testing before the implementation of 1.3.6, printed results of integers larger than 16 digits would have a randomised selection of integers past the 16th digit. This was of concern due to its potential impact on normal calculations. However, upon further testing and research, this is primarily due to the limited precision, especially when printing in C.

```
Enter choice: 1
Enter equation:  1797693134862315708145274237317043567980705675258449965989174768031572607800285387605895586327668781715
404589535143824642343213268894641827684675467035375169860499105765512820762454900903893289440758685084551339423045832369
0322294816580855933212334827479782620414472316873817718091929988125040402618412485836 8
Equation: 1797693134862315708145274237317043567980705675258449965989174768031572607800285387605895586327668781715404589 5
3514382464234321326889464182768467546703537516986049910576551282076245490090389328944075868508455133942304583236902322 94
816580855933212334827479
Token: 1797693134862315708145274237317 Type: 0
[Mode: Simple Calculator]

=== AST ===
Number: 1797693134862315795899186413568.000000
Result: 1797693134862315795899186413568.000000
```

This led to the three possible choices of **(1) limiting each numerical input** to have a max of 16-digits or **(2) truncating the visible output** to 16-digits or **(3) changing the visible output to scientific notation**. (3) was chosen over (1) due to the possibility of overflow issues, leading back to the same problem. (2) was possible but not as recommended due to less accurate visual feedback.

`Enter choice: 1`
 	
`-------------------------`
`=== SIMPLE CALCULATOR ===`
`-------------------------`
`Enter equation: 1234567890123456`
 	
`Equation: 1234567890123456`
 	
`=== Tokenizer ===`
`Token: 1234567890123456 Type: 0`
 	
`=== AST ===`
`Number: 1234567890123456.000000`
 	
`Result: 1234567890123456.000000`
	`Enter choice: 1`
	
	`-------------------------`
	`=== SIMPLE CALCULATOR ===`
	`-------------------------`
	`Enter equation: 12345678901234567`
	
	`Equation: 12345678901234567`
	
	`=== Tokenizer ===`
	`Token: 12345678901234567 Type: 0`
	
	`=== AST ===`
	`Number: 1.23457e+16`
	
	`Result: 1.23457e+16`

Input of 16-digits long integer	Input of 17-digits long integer

1.4. Compatibility Measures

To ensure cross-platform compatibility, the project was designed to support execution on macOS, Linux, and Windows environments. On macOS and Linux systems, the $make$ command is available by default, allowing users to compile the program directly using the Makefile.

However, on Windows systems, the $make$ command is not natively supported, which results in an error indicating that the command is not recognised. To address this issue, we utilised $mingw32 - make$ which is included as part of the MinGW environment and serves as an equivalent to the $make$ utility.

As a result, Windows users are required to execute the command $mingw32 - make$ instead of $make$ to successfully compile the project. This approach ensures that the same Makefile can be used across different operating systems with minimal modifications.

2. Constraint Satisfaction

2.1. File Reading/Writing

The project requirements state that **"our system should process files as input and output files at the end/during the process"**.

These constraints were satisfied in Section 1.3.3. Input and 1.3.7. Graph, apart from standard inputs and outputs, **(1) text files** (specifically CSV) are **accepted as input** if the programme is run with an additional argument for the CSV file, and **(2) binary files** (specifically PNG) can be **outputted** if 'Graph Mode' is selected.

2.2. Parser

Another project requirement states that "**our own parser is to be created, with it being robust and able to detect incorrect input"**.

This constraint is satisfied within sections 1.3.4. Lexer (Tokenizer) and 1.3.5. Parser, with the lexer being used to perform lexical analysis and the CFG and AST within the latter section used to perform syntax and semantic analysis. Furthermore, input for choice selection was also made to be robust and is addressed within 1.3.9.2. Choice Input.

2.3. State Machine

The project does not use an explicit state machine for calculations, but rather replaces the FSM with the use of ASTs for efficiency. However, a simple "state machine" is performed using if-else statements within main.c for choice selection to select the different modes.

2.4. External Libraries

Another constraint was **"No external libraries, except for specific cases"**. This is satisfied where most libraries used are part of the standard C library, with the exception of the use of a third-party library mentioned in 1.3.7. Graph for graphing purposes.

Below is a list of all the libraries used:

- Standard libraries: stdlib.h, string.h, stdio.h, float.h, limits.h
- Third-party: pbPlots/pbPlots.h, pbPlots/supportLib.h

2.5. C Programming Principles & Practices

2.5.1. Memory Management

Since there are a lot of components and modules where memory is dynamically allocated during runtime due to dependency on user input, these were ensured to be properly freed once the calculator has outputted the required results. Some examples include the ASTNode typed variables as well as some of the graph variables.

2.5.2. Modularity & Reusability

For specific use cases, a good practice is to section out code into multiple C files alongside their header files. Implementing their respective header files allows for the reusability of functions and libraries without having to redeclare them as a function within a different file. Furthermore, these C files are limited to certain functions for readability (e.g. ast.c for ASTs, dualUtils.c for differentiation…etc).

2.5.3. Code Practices

2.5.3.1. Don't Repeat Yourself (DRY)

Repeated lines of code were encapsulated in their own functions. A prime example of where it is used is within main.c, where the different modes were made limited to the functions $simpleCalc()$, $differentiation()$, $showGraphs()$ were repeated in other instances such as "File mode" and "Testing", separated to include additional functionality based on their respective use cases.

2.5.3.2. Single Responsibility Principle (SRP)

This principle advocates limiting functions to having specific purposes, which is constantly repeated throughout our code. Furthermore, this was also satisfied across files for modularity purposes as mentioned in 2.5.2. Modularity.

2.5.4. C Project Practices

2.5.4.1. Makefile

To ensure consistency within compilations on different operating systems, as mentioned in 1.4. Compatibility Measures ,this was implemented and commented for readability to allow for easy changes.

2.5.4.2. README

The README file was focused on summarising the main purpose of the project, system architecture and setup instructions to give the lay user a clear and straightforward overview without delving too deep into its implementation.

3. Challenges

3.1. Finite State Machine (FSM)

Initially, we began with designing a Finite State Machine (FSM). However, we faced difficulties in determining which operators and functions should be supported. We also considered creating separate FSMs for different modes, but found that these designs were largely repetitive and inefficient. As a result, we decided to retain a unified FSM design that incorporates all the required states for the graphing calculator.

3.2. Context Free Grammar (CFG)

One of the key challenges in developing the graphing calculator was designing a context-free grammar (CFG) capable of accurately parsing mathematical expressions. In the initial draft of the CFG, we identified several issues, such as left recursion and a lack of alignment with the iterative parsing strategy used in the implementation. To address these problems, we reconstructed the CFG by introducing helper non-terminals and reconstructing the grammar to eliminate ambiguity and improve compatibility with the parser.

3.3. Abstract Syntax Tree (AST)

Another challenge was the construction and management of the Abstract Syntax Tree (AST). Translating parsed expressions into a well-structured AST was non-trivial, as each node needed to correctly represent the hierarchy and relationships between operands and operators. Errors in the AST structure often led to incorrect evaluation results, making debugging particularly challenging. To address this, we refined the AST construction by establishing a clear mapping between grammar rules and parsing functions, ensuring consistency between the CFG and the AST. Operators were represented as internal nodes, while operands such as numbers and variables were represented as leaf nodes, preserving the hierarchical structure of expressions. Recursive parsing was used to correctly handle nested expressions and enforce operator precedence. In addition, validation checks and error handling were incorporated during parsing to detect and resolve issues early. Through iterative refinement, we achieved a more stable and accurate AST, improving the overall correctness of the graphing calculator.

3.4. Third-party Library for Graph in C

Looking for a suitable third-party library to support the graph feature in our program proved to be a challenge, as there were limited choices and the majority of them were incompatible with our use case by being unnecessarily complex. Before deciding on using pbPlots, there were several others considered. One such other option was GNUPlot but this required an external program to be installed to be used. This is not suitable as we don’t expect each user to have to install this program just to use the graph feature. Other than that, due to the usage of pbPlot, we had to change the standard of the compiling the program from c90 to c99 to be compatible.

4. Conclusion

This project was intended to design and implement a graphing calculator in C capable of parsing, evaluating, and visually plotting mathematical expressions. Through the complete flow of the pipeline of input handling, lexical analysis, parsing, evaluation and graph rendering, we successfully achieved the aim of the project.

The final system supports 3 functional modes: a simple calculator for numerical expressions, a differentiation mode combining both symbolic and automatic differentiation via dual numbers and a graph plotting mode that dynamically adjusts axis ranges of x and y limits based on the input expression.

The project gave us a hands-on appreciation for how programming language theory applies in practice. Designing and refining our CFG allowed us to see the limitations posed by implementation issues like eliminating left recursion. Building AST from scratch reinforced the value of separating parsing from evaluation, making the system easier to extend and debug. Implementing symbolic differentiation and exploring dual numbers for automatic differentiation deepened our understanding of how mathematical abstractions can translate into efficient computational solutions. Working in C further increased our knowledge about low-level considerations like the floating-point representation and data verification. Also, building the project with compiler flags for pedantry and strictness was extremely helpful. Considering all warnings as compilation errors forced us to fix things that would usually go unnoticed, such as implicitly declared functions, unused variables, and non-standard code structure. In this case, we considered these flags not as limitations but rather as an automatic source code review process that helped to produce high-quality code.

Beyond the technical skills gained, this project strengthened our ability to decompose a complex problem into manageable components and collaborate effectively as a team. Overall, this project was a valuable exercise in bridging theory and practice. The hands-on experience of constructing a CFG, building an AST, and implementing a complete pipeline in C has given us a much deeper and concrete understanding of the core concepts covered in our classes.

5. Appendix / References

5.1. GitHub Link

https://github.com/STHSCare12/PLC_Graphing_Calculator.git

5.2. pbPlots GitHub Link

https://github.com/InductiveComputerScience/pbPlots

5.3. Table 1: Simple Calculation Test Cases

	TYPE	EQUATION	EXPECTED RESULT
ADDITIVE	ADDITION (+)	+1+2	3
		INT_MAX+1	INT_MAX+1 (depends on OS)
	SUBTRACTION (-)	-1-2	-3
		INT_MIN-1	INT_MIN-1 (depends on OS)
		1-(-1)	2
		1-(-(-(-(-1))))	0
	COMBINED	INT_MAX+INT_MIN	-1
		-1+-1	-2
		(1+2)-(1+2)	0
		(1+2)-(-(1+2))	6
MULTIPLICATIVE	MULTIPLY (*)	*1	0
		INT_MAX*0	0
		INT_MAX*-1	-INT-MAX
		INT_MIN*-1	-INT_MIN
		INT_MIN*0	0
		(1*0)*(1*-1)	0

	DIVIDE (/)	2/1	2
		/0	nan
		/1	0
		2/2	1
		2/-1	-2
		DIV_6, "1/(1/0)", 1.0/(1.0/0.0), TEST_S	nan
	COMBINED (/*)	2*4/2	4
POWER	POWER (^)	^2	0
		INT_MAX^INT_MAX	INFINITY
		INT_MAX^0	1
		-1^2	-1
		(-1)^2	1
		1^-2	1
		-1^-2	-1
		-2^4	-16
		-2^-4	-0.0625
COMBINED	COMBINED (+-/*)	2*2+2/2-2	3
		2*(2+2-2)/2	2

5.4. Table 2: Differentiate Test Cases

TYPE	EQUATION	EXPECTED RESULT (x=2)
MULTIPLY (*)	1*x*1*x	4
	(1*x)*(1*x)	4
	-1*x*-1*x	4
	-1*x*1*x	-4
DIVIDE (/)	2*x/2*x	4
	-x/-x	0
	x/2	0.5
	x/-x	0
POWER (^)	x^2	4
	-x^2	-4
	x^x	pow(2,2)*(log10(2)+1) (Using C math library)
TRIGO	sin(x)	cos(2*(M_PI/180))
	sin(-x)	-cos(2*(M_PI/180))
	cos(x)	-sin(2*(M_PI/180))
	cos(-x)	sin(-2*(M_PI/180))
	tan(x)	1/(pow(cos(2*(M_PI/180)), 2))
	tan(-x)	-(1/(pow(cos(2*(M_PI/180)), 2)))
	exp(x)	exp(2)
	exp(-x)	-exp(-2)
	log(x)	1/(2*log(10))
	log(-x)	1/(2*log(10))